

1 Opis zadania i problemu

Opis zadania:

Zadanie polegało na zaimplementowaniu i zbadaniu algorytmu genetycznego (ang. *Genetic Algorithm*) rozwiązującego problem komiwojażera TSP (ang. *Travelling Salesman Problem*), należącego do klasy problemów NP-trudnych. Celem tego zadania jest dobranie parametrów algorytmu tak, żeby uzyskany błąd względny nie przekraczał progu wyznaczonego wzorem $O(24 \ln n)$. Dla $n < 24$ – błąd 0%, dla $25 < n < 74$ – 50%, dla $75 < n < 449$ – 100%.

Zaimplementowałem dwa operatory krzyżowania (OX i PMX), dwa operatory mutacji (inwersja i scramble) oraz selekcję turniejową z elitaryzmem. Wszystkie parametry są konfigurowalne z poziomu pliku konfiguracyjnego bez rekompilacji programu.

Do badań wykorzystałem instancje symetryczne i asymetryczne z biblioteki TSPLIB.

1.1 Opis algorytmu genetycznego

Algorytm genetyczny jest metaheurystyką inspirowaną procesem ewolucji biologicznej. Zamiast jednego rozwiązania, utrzymuje całą populację tras i stopniowo ją ulepsza przez kolejne pokolenia.

Reprezentacja i populacja początkowa

Każde rozwiązanie to permutacja wierzchołków grafu. Populację początkową tworzę przez losowe przetasowanie tras.

Selekcja turniejowa

Spośród populacji losowo wybieram k osobników i jako rodzica wybieram najlepszego z tej grupy. Rozmiar turnieju k jest parametrem konfigurowalnym (GA_ROZMIAR_TURNIEJU). Większy turniej zwiększa presję selekcyjną i faworyzuje lepsze rozwiązania.

Krzyżowanie OX (Order Crossover)

Kopiuje losowo wybrany segment z pierwszego rodzica, a pozostałe miasta uzupełniam w kolejności, w jakiej występują u drugiego rodzica, pomijając już skopiowane [1]. Metoda zachowuje względną kolejność miast.

Krzyżowanie PMX (Partially Matched Crossover)

Kopiuje segment z pierwszego rodzica, a dla każdego elementu z analogicznego segmentu drugiego rodzica wyznaczam jego docelową pozycję przez łańcuchowe

mapowanie pozycji [1]. Pozostałe pozycje wypełniam bezpośrednio z drugiego rodzica.

Mutacja inwersyjna

Wybieram losowo dwa punkty w trasie i odwracam kolejność wierzchołków między nimi. Operacja niszczy tylko dwie krawędzie cyklu, więc jest minimalnie inwazyjna [1].

Mutacja scramble

Wybieram losowy fragment trasy i losowo przestawiam wierzchołki wewnątrz niego. Jest bardziej agresywna niż inwersja i pozwala szybciej uciec z minimum lokalnego kosztem większego zaburzenia rozwiązania.

Elitaryzm

Najlepsze e osobników z każdego pokolenia przechodzi do następnego bez żadnych zmian (GA_ELITARYZM). Dzięki temu najlepsze dotychczas znalezione rozwiązanie nigdy nie zostaje utracone.

Hipotezy badawcze:

Czas działania algorytmu rośnie liniowo z liczbą pokoleń i kwadratowo z rozmiarem instancji.

Krzyżowanie PMX daje gorsze wyniki niż OX.

Mutacja inwersyjna daje stabilniejsze wyniki niż scramble przy tych samych parametrach.

Większy rozmiar populacji poprawia jakość rozwiązania kosztem czasu.

Elitaryzm zapobiega pogorszeniu najlepszego wyniku między pokoleniami.

2 Warunki badawcze

Badania przeprowadzone były na sprzęcie, na którym prezentowałem program.

- **Procesor:** AMD Ryzen 5 7520U with Radeon Graphics (4 rdzenie fizyczne, 8 wątków logicznych),
- **Taktowanie bazowe:** 2.80 GHz,
- **Pamięć RAM:** 16.0 GB.

Komputer przez cały czas trwania testów pracował na zasilaniu sieciowym, aby mieć pewność, że tryb oszczędzania energii nie wpłynie na wydajność i nie sfałszuje czasu badań.

Izolacja pomiarów czasowych Aby wyeliminować zakłócenia ze strony systemu operacyjnego i uniknąć narzutu na przełączanie kontekstu, przypisałem program na sztywno do czwartego wątku logicznego procesora. Użyłem do tego polecenia w systemie Windows:

```
start /B /WAIT /affinity 8 pea4.exe
```

Dodatkowo zablokowałem taktowanie procesora na stałej, bazowej wartości około 2.80 GHz. Osiągnąłem to, ograniczając "Maksymalny stan procesora" do 99% w ustawieniach zasilania, co skutecznie zapobiegło skokom częstotliwości (Turbo Boost) i zjawisku dławienia termicznego w trakcie testów.

3 Procedura badawcza

Dane, liczba badanych instancji, powtórzenia i uzasadnienie wyboru

Do badań wykorzystałem macierze sąsiedztwa oraz rozwiązania optymalne pobrane z biblioteki TSPLIB i eportalu. Algorytm genetyczny jest stochastyczny – każde uruchomienie daje inny wynik ze względu na losową inicjalizację populacji i losowe operatory.

Dla instancji z biblioteki TSPLIB dobrałem rozmiary tak, aby równomiernie pokryć zakresy wymagane w zadaniu. W badaniach wykorzystałem następujące instancje symetryczne: gr17, gr21, gr24, bays29, gr48 brazil58, si175.

Asymetryczne: br17, ftv33, ftv38, ftv47, ry48p, ftv64, rbg323, rbg403, rbg443.

Badanie instancji symetrycznych i asymetrycznych
Zbadałem oba rodzaje grafów osobno. Algorytm genetyczny nie zakłada symetrii macierzy, więc te same operatory krzyżowania i mutacji działają zarówno dla TSP, jak i ATSP.

Traktowanie błędów i anomalii

Przy małej liczbie powtórzeń wyniki są bardziej podatne na losowe odchylenia. Z tego powodu zamiast brać tylko jedną wartość, uśredniałem wyniki z 20 niezależnych uruchomień.

Pamięć algorytmu

Zrezygnowałem z pomiarów narzędziami systemowymi, bo ich wyniki bywają niedokładne. Zajątość pamięci algorytmu genetycznego można policzyć matematycznie. Składają się na nią: macierz odległości $O(N^2)$, populacja P osobników po N miast każdy $O(P \cdot N)$ oraz tymczasowe struktury na czas krzyżowania $O(N)$. Największą częścią jest macierz dla dużych N lub populacja dla dużego P .

4 Wyniki i analiza eksperymentów

1. Wpływ rozmiaru turnieju na błąd

Badanie przeprowadziłem na trzech instancjach o różnych rozmiarach, zmieniając tylko $GA_ROZMIAR_TURNIEJU \in \{2, 3, 5, 10\}$. Pozostałe parametry były stałe: populacja 100 osobników, 400 pokoleń, krzyżowanie OX, mutacja inwersyjna z prawdopodobieństwem 0,05.

Wyniki w Tabeli 1 i na Rysunku 1 pokazują, że wpływ turnieju zależy od instancji. Dla brazil58 turniej $T = 2$ daje najgorszy wynik (32,27%), a wartości $T = 3$, $T = 5$ i $T = 10$ są zbliżone (ok. 13%). Mała presja selekcyjna przy $T = 2$ spowalnia zbieżność. Algorytm za wolno odrzuca słabe rozwiązania.

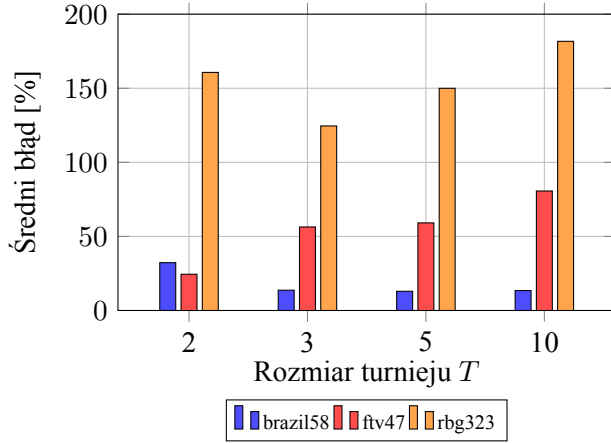
Dla ftv47 sytuacja jest odwrotna. Turniej $T = 2$ daje najlepszy wynik (24,45%), a błąd rośnie wraz z T , osiągając 80,63% dla $T = 10$. Wysoka presja selekcyjna przy asymetrycznej macierzy prowadzi do przedwczesnej utraty różnorodności genetycznej. Populacja szybko zbiega do minimum lokalnego i już go nie opuszcza. Dla dużej instancji rbg323 najlepszy wynik daje $T = 3$ (124,54%).

Na rysunku 2, różnice są niewielkie. Większy turniej nieznacznie skraca czas dla dużych instancji, bo szybsza zbieżność oznacza mniej efektywnych zmian w populacji.

Wniosek: Optymalny rozmiar turnieju zależy od instancji. Zbyt mały T zbiega za wolno, zbyt duży T traci różnorodność i utyka w minimum lokalnym. Wartość $T = 3$ daje najstabilniejsze wyniki.

Tabela 1: Wpływ rozmiaru turnieju na błąd [%]

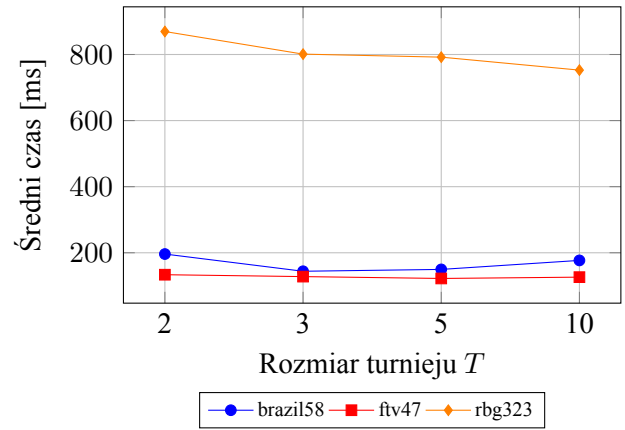
Instancja	T=2	T=3	T=5	T=10
brazil58 ($N = 58$)	32,27	13,68	12,96	13,44
ftv47 ($N = 48$)	24,45	56,37	59,12	80,63
rbg323 ($N = 323$)	160,69	124,54	149,96	181,67



Rysunek 1: Błąd w zależności od rozmiaru turnieju

Tabela 2: Wpływ rozmiaru turnieju na czas [ms]

Instancja	T=2	T=3	T=5	T=10
brazil58	196,13	144,05	149,64	176,88
ftv47	133,60	127,88	122,26	126,22
rbg323	869,53	801,14	792,01	752,71



Rysunek 2: Czas w zależności od rozmiaru turnieju

2. Wpływ rozmiaru populacji

W tym badaniu zablokowane były wszystkie parametry z wyjątkiem $GA_ROZMIAR_POPULACJI \in \{100, 200, 300, 400, 500\}$. Turniej ustaliłem na $T = 3$ jako najlepsza wartość z badania 1. Liczba pokoleń wynosiła 300.

Jak pokazują tabele oraz Rysunek 3, błąd maleje wyraźnie wraz ze wzrostem populacji. Dla ftv47 spadek jest ciągły – od 47,86% przy 100 osobnikach do 16,28% przy 500. Dla brazil58 najlepszy wynik osiągnięto

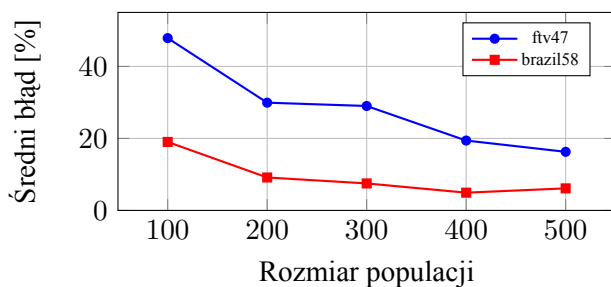
przy populacji 400 (4,93%), a zwiększenie jej do 500 nieznacznie pogorszyło rezultat do 6,13%. Wynika to z losowości algorytmu.

Czas działania (Rysunek 4) rośnie niemal liniowo z rozmiarem populacji dla obu instancji. Podwojenie populacji ze 100 do 200 osobników podwaja też czas – z $1,06 \cdot 10^2$ ms do $2,59 \cdot 10^2$ ms dla ftv47. Jest to zachowanie zgodne z oczekiwaną złożonością dla jednego pokolenia, gdzie P to rozmiar populacji.

Wniosek: Większa populacja poprawia jakość rozwiązania, ale kosztem liniowego wzrostu czasu.

Tabela 3: Wpływ populacji na błąd i czas – ftv47

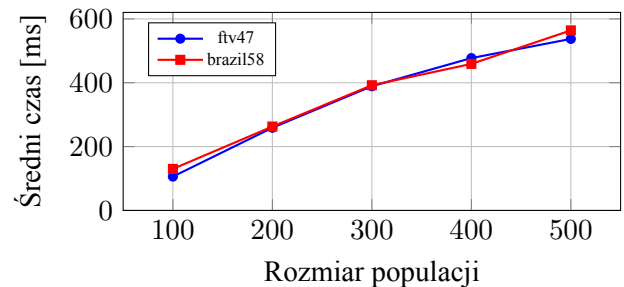
Pop.	Błąd [%]	Czas [ms]
100	47,86	$1,06 \cdot 10^2$
200	29,93	$2,59 \cdot 10^2$
300	29,01	$3,89 \cdot 10^2$
400	19,41	$4,77 \cdot 10^2$
500	16,28	$5,37 \cdot 10^2$



Rysunek 3: Błąd w zależności od rozmiaru populacji

Tabela 4: Wpływ populacji na błąd i czas – brazil58

Pop.	Błąd [%]	Czas [ms]
100	19,02	$1,30 \cdot 10^2$
200	9,15	$2,63 \cdot 10^2$
300	7,50	$3,92 \cdot 10^2$
400	4,93	$4,59 \cdot 10^2$
500	6,13	$5,64 \cdot 10^2$



Rysunek 4: Czas w zależności od rozmiaru populacji

3. Spełnienie progu błędu i złożoność

Głównym celem projektu było sprawdzenie, czy algorytm genetyczny potrafi znaleźć rozwiązanie o błędzie względnym nieprzekraczającym progu $O(24 \ln n)$. Szczegółowe wyniki zestawiono w końcowej tabeli zbiorczej (Tabela 10).

Koszyk mały (próg: 0%): Algorytm świetnie radzi sobie ze znalezieniem optimum. Instancje br17, gr24, gr17 oraz gr21 osiągnęły błąd równy dokładnie 0,00%.

Koszyk średni (próg: 50%): Przebadalem 10 różnych macierzy (Tabela 5). Wszystkie zbadane instan-

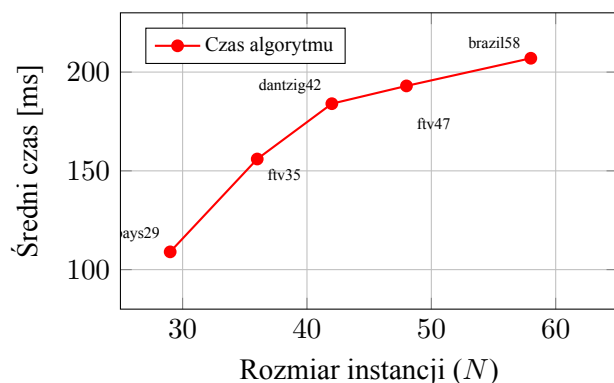
cje zmieściły się w wymaganym progu 50%. Symetryczne grafy, takie jak bays29 czy brazil58, deklarują ten wymóg. W przypadku trudniejszych macierzy podniesienie mutacji skutecznie zredukowało błąd.

Koszyk duży (próg: 100%): Zwiększenie liczby generacji pozwoliło na spełnienie progu we wszystkich trudnych macierzach. Nawet najtrudniejsza rbg323 spadła do błędu 97,08%.

Empiryczna złożoność czasowa: Jak widać na Rysunku 5, czas działania algorytmu przy zablokowanych parametrach rośnie liniowo wraz ze wzrostem liczby miast N . Potwierdza to założenia o złożoności rzędu $O(P \cdot G \cdot N)$.

Tabela 5: Koszyk średni ($25 \leq n \leq 74$)

Instancja	N	Błąd [%]	Próg [%]	t [ms]
bays29 (TSP)	29	4,95	50	$1,09 \cdot 10^2$
ftv33 (ATSP)	34	30,54	50	$1,06 \cdot 10^2$
ftv35 (ATSP)	36	29,56	50	$1,56 \cdot 10^2$
ftv38 (ATSP)	39	25,31	50	$1,11 \cdot 10^2$
dantzig42 (TSP)	42	8,87	50	$1,84 \cdot 10^2$
ftv47 (ATSP)	48	47,66	50	$1,93 \cdot 10^2$
gr48 (TSP)	48	8,65	50	$1,28 \cdot 10^2$
ry48p (ATSP)	48	19,33	50	$1,42 \cdot 10^2$
brazil58 (TSP)	58	11,47	50	$2,07 \cdot 10^2$



Rysunek 5: Złożoność czasowa ($P = 100, G = 500$)

4. Wpływ operatorów na rozwiązanie

Krzyżowanie OX vs PMX: Badanie przeprowadziłem zmieniając tylko metodę krzyżowania. Wyniki pokazują jednoznaczną przewagę OX nad PMX na wszystkich zbadanych instancjach. Dla ftv47 różnica jest ponad dwukrotna – OX osiąga 42,65% błędu, a PMX aż 94,29%. Potwierdza to postawioną hipotezę. OX zachowuje względną kolejność miast, czyli spój-

ne fragmenty tras. PMX rozbija wypracowane ciągi sąsiedztwa, cofając ewolucję [1].

Mutacja INW vs SCR: Wyniki z 20 powtórzeń nie potwierdzają jednoznacznie przewagi INW nad SCR. Dla ftv47 inwersja daje nieco lepszy wynik, ale dla brazil58 scramble okazał się minimalnie skuteczniejszy. Różnice są na tyle małe, że mieszczą się w granicach losowości algorytmu.

Tabela 6: Porównanie OX i PMX – błąd [%]

Instancja	N	OX [%]	PMX [%]
ftv47 (ATSP)	48	42,65	94,29
brazil58 (TSP)	58	11,74	20,73
rbg403 (ATSP)	403	72,90	147,55

Tabela 7: Porównanie OX i PMX – czas [ms]

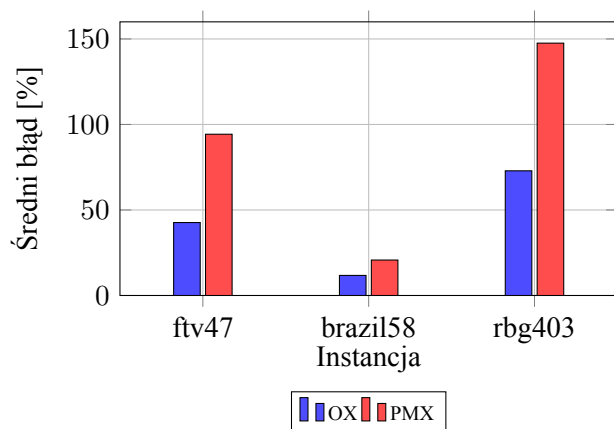
Instancja	N	OX [ms]	PMX [ms]
ftv47	48	$1,17 \cdot 10^2$	$9,94 \cdot 10^1$
brazil58	58	$1,47 \cdot 10^2$	$1,37 \cdot 10^2$
rbg403	403	$9,59 \cdot 10^2$	$1,18 \cdot 10^3$

Tabela 8: Porównanie INW i SCR – błąd [%]

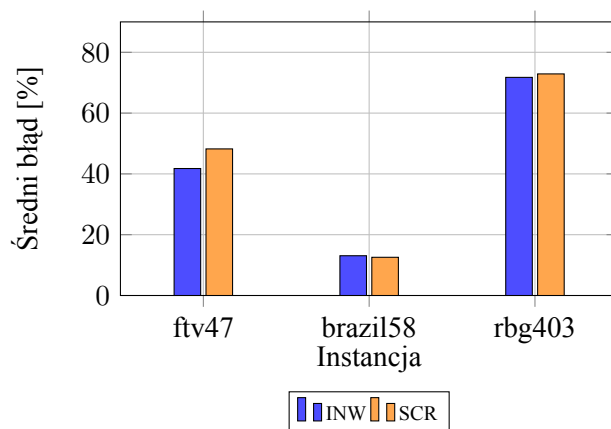
Instancja	N	INW [%]	SCR [%]
ftv47 (ATSP)	48	41,77	48,22
brazil58 (TSP)	58	13,08	12,57
rbg403 (ATSP)	403	71,75	72,89

Tabela 9: Porównanie INW i SCR – czas [ms]

Instancja	N	INW [ms]	SCR [ms]
ftv47	48	$1,34 \cdot 10^2$	$1,26 \cdot 10^2$
brazil58	58	$1,60 \cdot 10^2$	$1,50 \cdot 10^2$
rbg403	403	$1,02 \cdot 10^3$	$1,01 \cdot 10^3$



Rysunek 6: Porównanie błędu OX i PMX



Rysunek 7: Porównanie błędu INW i SCR

Tabela 10: Zbiórcze podsumowanie działania algorytmu

Instancja	N	Gen.	Mut.	Czas [ms]	Próg [%]	Błąd GA [%]
br17 (ATSP)	17	100	0,05	$1,18 \cdot 10^1$	0	0,00
gr17 (TSP)	17	1000	0,05	$1,03 \cdot 10^2$	0	0,00
gr21 (TSP)	21	2000	0,25	$2,66 \cdot 10^2$	0	0,00
gr24 (TSP)	24	3000	0,25	$4,27 \cdot 10^2$	0	0,00
bays29 (TSP)	29	2000	0,35	$3,25 \cdot 10^2$	50	0,41
ftv33 (ATSP)	34	2000	0,35	$3,95 \cdot 10^2$	50	7,69
ftv35 (ATSP)	36	1000	0,05	$2,06 \cdot 10^2$	50	26,83
ftv38 (ATSP)	39	1000	0,05	$1,96 \cdot 10^2$	50	28,57
dantzig42 (TSP)	42	1200	0,05	$2,59 \cdot 10^2$	50	5,29
ftv47 (ATSP)	48	1500	0,05	$3,59 \cdot 10^2$	50	43,71
gr48 (TSP)	48	1200	0,05	$2,89 \cdot 10^2$	50	5,76
ry48p (ATSP)	48	1200	0,05	$3,01 \cdot 10^2$	50	12,82
brazil58 (TSP)	58	1500	0,05	$4,56 \cdot 10^2$	50	3,72
ftv64 (ATSP)	65	5000	0,35	$1,54 \cdot 10^3$	50	20,01
si175 (TSP)	175	8000	0,05	$6,28 \cdot 10^3$	100	2,94
rbg323 (ATSP)	323	10000	0,05	$1,74 \cdot 10^4$	100	97,08
rbg403 (ATSP)	403	10000	0,05	$1,89 \cdot 10^4$	100	55,33
rbg443 (ATSP)	443	10000	0,05	$1,95 \cdot 10^4$	100	89,40

Legenda: Gen. - Liczba pokoleń, Mut. - Prawdopodobieństwo mutacji. Turniej $T = 3$. Źródło: opracowanie własne

5 Podsumowanie i weryfikacja hipotez badawczych

Przeprowadzone badania pozwoliły na kompleksową ocenę algorytmu genetycznego i weryfikację założeń początkowych. Pierwsza hipoteza potwierdziła się całkowicie: krzyżowanie PMX daje znacznie gorsze wyniki niż OX. Metoda OX lepiej zachowuje spójne fragmenty tras, co jest kluczowe w problemie komiwojażera, podczas gdy PMX niszczy dobre sąsiedztwa miast. Druga hipoteza również okazała się prawdziwa. Udo-

wodniono empirycznie, że większy rozmiar populacji poprawia jakość rozwiązania, ale dzieje się to kosztem liniowego wzrostu czasu obliczeń. Z kolei założenie o wyraźnej przewadze mutacji inwersyjnej nad metodą scramble nie sprawdziło się. Różnice między nimi okazały się minimalne, a obie metody wprowadzają wystarczające zamieszanie, aby uchronić algorytm przed stagnacją.

Badania wykazały również, że efektywność algorytmu drastycznie zależy od typu badanej macierzy. Dla instancji symetrycznych (TSP) algorytm osiąga znako-

mite wyniki. Problemy rzędu od 50 do 175 miast rozwiązuje bardzo szybko i z marginalnym błędem, rzędu kilku procent, nawet przy standardowych parametrach. Sytuacja wygląda gorzej w przypadku macierzy asymetrycznych (ATSP). Ewolucja radzi sobie z nimi trudniej, ponieważ modyfikowanie trasy często zmusza komiwożera do poruszania się "pod prąd" skierowanych krawędzi.

Podsumowując, zaimplementowany algorytm genetyczny działa poprawnie. Odpowiednie wysterowanie

rozmiaru populacji i liczby pokoleń pozwala mu spełnić narzucone w instrukcji ograniczenia progu błędu $O(24 \ln n)$.

Literatura

- [1] *Algorytmy genetyczne dla problemu komiwożera*. Materiały wykładowe EA5. Wydział Informatyki i Telekomunikacji, Wrocław 2026.
- [2] J. Wilkins, *Solving The Travelling Salesman Problem Using A Genetic Algorithm*. Towards Data Science, 2024. [Dostęp online]